

Data Engineer Test Assignment

The purpose of this assignment is to evaluate your ability to both establish structure and implement elements for a data pipeline. It has been designed to reflect real-world situations, where requirements can often change, and the initial tasking may lack detail. We do not expect a fully production-ready solution; however, we are excited to see your ideas and implementation that show your thinking.

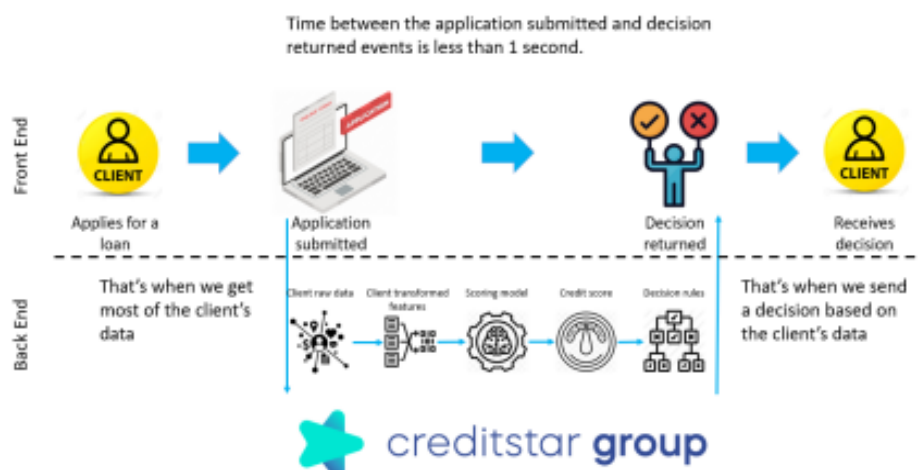
Expected time: 3-6 hours. We value working on core functionality over polished completeness. State assumptions and document what you'd do with more time rather than trying to build everything.

As a data engineer, one of your responsibilities will be to contribute to the development of our in-house data services. One of them is data streaming from replicas of operational databases to object storage and making it available for other data services, such as machine learning models and decision engines. The following task reflects one of the problems that our data engineering team faces.

Business case

- We need to automate decisions made on different steps of different business processes.
- Your task is to help automate accept/reject decisions in the loan application flow.
- The client applies for a loan by submitting an online application.
- Submitted client's raw data is stored in Creditstar CRM.
- A decision automation service is called by the CRM:
 - The client's raw data is loaded to the service.
 - Client's raw data is transformed into features.
 - Features are sent to a scoring model endpoint.
 - The scoring model returns a credit score value to the service.
 - Credit score and other features are run through decision rules to make a decision.
 - Decision value is sent back to the CRM.
- CRM acts on the decision returned by a decision automation service.
- Credit decision (approve/reject) is communicated to the client.

Decision automation in application flow



Your Task

Restore the given database dump file into a Postgres database server and stream the tables into object storage (e.g., S3, MinIO, or equivalent). Your pipeline should detect new or changed data in the database and automatically stream it, along with computed features, to object storage.

As a team, we are implementing a streaming service that will be used as an ingestion to the feature store. It will be the foundation of our decision engine services, which will consume stored features for the decision-making process. Decision services will be triggered as soon as a customer applies for a loan by submitting their data, so fetching data from the operational database to the feature store via a streaming service must be reliable and near real-time.

These stored features will also be consumed by data scientists to train and develop machine learning models.

Models and decision engine service require derived/transformed features, so the streaming pipeline should be able to handle transformations too. The transformation logic should be implemented in Python, meant to avoid double work done by team members and reduce the time it takes to develop a feature from idea to production.

Sub Task 1:

Set up the infrastructure needed to run your streaming pipeline. You're free to choose any approach - local tools, cloud free tiers, or a mix. We don't care about the production-readiness of the infra.

We care about:

- Why did you choose what you chose
- What tradeoffs did you consider
- How would you do it differently with more budget/time.

Sub Task 2:

Data scientists requested the following features in the streaming pipeline:

- `client.paid_loans.count` - number of previously paid loans for this client.
- `client.days_since_last_late_payment.count` - count of days since the last late payment made by this client.
- `client.profit_in_last_90_days.rate` - sum of interest payment amount received from loans issued in the last 90 days for this client divided by sum of loan amount issued in the last 90 days for this client.

Implement the calculation logic for these features in Python and stream them to the object storage using the solution developed in Sub Task 1. Document your assumptions about edge cases (e.g., client with no late payments, no loans in the last 90 days).

Submission:

Share your solution via a git repository of your preference with a README explaining how to run it.